Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ciencia de la Computación
IIC2233 — Programación Avanzada 2026-2

Publicación: 15 de octubre de 2026
Actividad evaluada

Actividad 6
Serialización y Networking 2

Entrega

- **Lugar:** Repositorio personal de GitHub — Carpeta: `Actividades/AC6`
- **Fecha máxima de entrega:** 15 de octubre a las 17:10 hrs
- **Ejecución de actividad**: La Actividad será ejecutada **únicamente** desde la terminal del computador. **Los *paths* relativos utilizados en la Actividad deben ser coherentes con esta instrucción, y no pueden modificarse.**

Importante: Antes de comenzar, comprueba que Git este funcionando correctamente en tu repositorio privado. Para esto, **sube los archivos base de la actividad de inmediato** (*add*, *commit*, *push*). Se espera que en esta actividad (así como en las demás actividades y tareas) utilices Git a lo largo de **todo tu desarrollo** como una herramienta, no sólo como un método de entrega. Es por esto que recomendamos enfáticamente que vayas subiendo tus cambios constantemente (***push***), ya que **problemas de último minuto** relacionados con la entrega y Git **no serán considerados**.

Instrucciones

Siguiendo con los juegos de adivinanzas, implementa un juego con cliente y servidor que se comuniquen mediante *sockets* y envíen mensajes cifrados mediante el manejo de *bytes*.

El juego consiste en adivinar el orden de una secuencia de números. Al inicio de la partida el juego definirá el orden de una secuencia y de ahí en adelante los jugadores tratarán de adivinarlo. Los jugadores tendrán múltiples intentos para adivinar el orden de la secuencia y el juego solo terminará cuando uno de ellos adivine el orden correcto. En cada uno de estos intentos, se le informará a los jugadores la cantidad de aciertos que tuvieron.

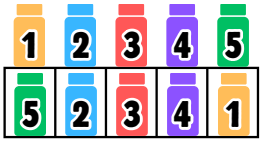


Figura 1: Ejemplo de la respuesta de un jugador (5 números de arriba) y la respuesta esperada (5 números de abajo). En este caso, el acierto es de 3 elementos.

El jugador interactuará desde la terminal, mientras que el servidor administrará el juego, los intentos de la partida y verificará las respuestas de cada jugador. Tanto el código del cliente como el del servidor se encontrarán en carpetas independientes, debes tratar cada una de estas carpetas como si fuese su propio computador.

Parte 1. Cifrado de mensajes

La comunicación entre el cliente y servidor será mediante *sockets* que utilice TCP como protocolo de transporte. Queda a tu criterio qué herramienta utilizar para transformar el mensaje a *bytes* y viceversa, y el protocolo para enviar los bytes a través de los *sockets*.

Cada mensaje que se envíe entre el cliente y el servidor deberá cifrarse en base a la siguiente fórmula:

```
(b + 0x2A) % 256
```

donde "`b`" corresponde a cada uno de los *bytes* a enviar. Para descifrar el mensaje, deberás invertir la operación realizada.

Deberás completar las funciones `cifrar_bytes_mensaje()` y `descifrar_bytes_mensaje()` que se encuentran definidas en el archivo `cifrado.py` de las carpetas del cliente y el servidor. Al final del archivo, encontrarás un "`if __name__ == '__main__'`" que te pondrá casos donde puedes probar tu código.

El contenido de este archivo es el mismo para el cliente y servidor, por lo que puedes copiar/pegar tu código una vez lo tengas listo.

Parte 2. Funcionamiento y reglas

Primeramente, deberás iniciar el servidor. Este deberá aceptar conexiones hasta que tenga 3 jugadores activos. Después, procederá a iniciar el juego.

Mientras el juego esté activo, el servidor deberá manejar los turnos de cada jugador y deberá asegurarse de jueguen todos los jugadores antes de que se repita uno de ellos.

Al inicio de cada turno se le deberá avisar a todos los jugadores el jugador que le toca jugar en ese momento. Posteriormente, dicho jugador deberá indicarle al servidor su respuesta. El servidor le indicará a todos los jugadores la cantidad de aciertos del jugador y avisará si el juego continúa o si fue completado. Si el juego continua, se inicia un nuevo turno; en caso contrario se finaliza el juego y la conexión con todos los clientes.

Todas las acciones del juego y sus respectivas validaciones ya se encuentran implementadas en la clase "`JuegoOrdenarNumeros`" del archivo "`servidor/juego.py`", revisa los *doc-strings* y el *type-hint* de cada método para comprender qué hacen.

Para visualizar el flujo completo del programa, tanto en el cliente como el servidor, recomendamos agregar *prints* con mensajes descriptivos de lo que está sucediendo en las distintas etapas del proceso. **El diseño y los textos son libres.**

A continuación se dejan los *prints* correspondientes a un servidor y uno de sus clientes. Usa estos ejemplos como guía.

Ejemplo en el Servidor

```
[SERVIDOR] Escuchando en ('127.0.0.1', 2233)

[SERVIDOR] Cliente ('127.0.0.1', 40636) conectado
[SERVIDOR] Cliente ('127.0.0.1', 46104) conectado
[SERVIDOR] Cliente ('127.0.0.1', 46120) conectado

[SERVIDOR] Juego iniciado. Respuesta correcta: 15342

[SERVIDOR] Turno de ('127.0.0.1', 40636)
[SERVIDOR] Esperando respuesta de ('127.0.0.1', 40636)
[SERVIDOR] ('127.0.0.1', 40636) indicó 25341
[SERVIDOR] ('127.0.0.1', 40636) acertó 3 de 5
[SERVIDOR] El juego continua

[SERVIDOR] Turno de ('127.0.0.1', 46104)
[SERVIDOR] Esperando respuesta de ('127.0.0.1', 46104)
[SERVIDOR] ('127.0.0.1', 46104) indicó 12345
[SERVIDOR] ('127.0.0.1', 46104) acertó 2 de 5
[SERVIDOR] El juego continua

[SERVIDOR] Turno de ('127.0.0.1', 46120)
[SERVIDOR] Esperando respuesta de ('127.0.0.1', 46120)
[SERVIDOR] ('127.0.0.1', 46120) indicó 54123
[SERVIDOR] ('127.0.0.1', 46120) acertó 0 de 5
[SERVIDOR] El juego continua

[SERVIDOR] Turno de ('127.0.0.1', 40636)
[SERVIDOR] Esperando respuesta de ('127.0.0.1', 40636)
[SERVIDOR] ('127.0.0.1', 40636) indicó 15342
[SERVIDOR] ('127.0.0.1', 40636) acertó 5 de 5
[SERVIDOR] ('127.0.0.1', 40636) resolvió el juego


[SERVIDOR] ('127.0.0.1', 40636) conexión cerrada
[SERVIDOR] ('127.0.0.1', 46104) conexión cerrada
[SERVIDOR] ('127.0.0.1', 46120) conexión cerrada
[SERVIDOR] Cerrando socket principal
```

Ejemplo en el Cliente

```
[CLIENTE ] Conectado en ('127.0.0.1', 40636)

[SERVIDOR] Turno de ('127.0.0.1', 40636)
[SERVIDOR] Es tu turno, envia tu respuesta
[CLIENTE ] > 15324
[SERVIDOR] Acertó 3 de 5
[SERVIDOR] Juego continua

[SERVIDOR] Turno de ('127.0.0.1', 46104)


[SERVIDOR] Acertó 2 de 5
[SERVIDOR] Juego continua

[SERVIDOR] Turno de ('127.0.0.1', 46120)


[SERVIDOR] Acertó 0 de 5
[SERVIDOR] Juego continua

[SERVIDOR] Turno de ('127.0.0.1', 40636)
[SERVIDOR] Es tu turno, envia tu respuesta
[CLIENTE ] > 15342
[SERVIDOR] Acertó 5 de 5
[SERVIDOR] Juego completado por ('127.0.0.1', 40636)
[CLIENTE ] Juego terminado
```

Notas

- No es necesario que tu código maneje casos de desconexión de usuarios o del servidor.
- No es necesario que tu código maneje casos en que el usuario entregue respuestas inválidas.
- Al momento de probar tu código, puedes trabajar con una menor cantidad de jugadores o números a adivinar, pero la versión final del código debe funcionar correctamente con 3 jugadores y una secuencia de 5 números.
- Al momento de definir el protocolo de envió de *bytes*, considera que las conexiones TCP pueden:
  - Fragmentar un mensaje en múltiples envíos.
  - Juntar en un envío múltiples mensajes independientes.

Puntaje por sección

Los criterios indican los aspectos principales que serán evaluados y pueden subdividirse durante la corrección.

Criterio	Puntaje
Parte 1.	
Es posible cifrar y descifrar mensajes siguiendo el protocolo descrito.	1,0 pts
Parte 2.	
El servidor acepta 3 jugadores correctamente. Una vez aceptados inicia el juego.	1,0 pts
En cada turno, el servidor avisa el jugador de la ronda, se recibe su respuesta y avisa la cantidad de aciertos del jugador.	1,0 pts
En cada turno, el cliente comprueba si es su turno. En caso de serlo, obtiene la respuesta del usuario y la envía al servidor.	1,0 pts
Si un jugador acierta todos los elementos del juego, el servidor avisa a todos los jugadores y cierra las conexiones con los clientes.	1,0 pts
Descuentos.	
Se podrá aplicar un máximo de **1 punto de descuento** por código con errores que impidan su ejecución, problemas de legibilidad, incumplimiento de PEP 8 o de las restricciones indicadas.	
Total	**5,0 pts**